Simple E-commerce Store

Setup & Database Guide

Django 5.2 | Python 3 | SQLite | Pillow
HTML5 | CSS3 | Vanilla JavaScript

Windows 11 | VS Code | Internship demo project
Demo store: no real payments are processed

Table of contents

2. Project overview and features 3
3. Hardware and software prerequisites 3
4. Python installation and verification 3
5. VS Code setup 4
6. Project folder structure 4
7. Virtual environment creation and activation 6
8. Installing Django, Pillow and other dependencies 6
9. SQLite database explanation 7
10. Django models and the ORM 7
11. Database migration instructions 8
12. Creating the administrator account 8
13. Loading sample products 8
14. Starting and accessing the website 9
15. Using the website as a customer 9
16. Using Django Admin as an administrator 9
17. Product image upload and media configuration 10
18. SQL examples (products, users, orders, order items) 10
19. How to use DB Browser for SQLite 12
20. Backing up and restoring the development database 12
21. Running the automated tests 13
22. Troubleshooting common errors 13
23. Windows Command Prompt vs PowerShell 14
24. Stopping and restarting the application 15
25. Security notes and limitations 15
26. Final verification checklist 16

2. Project overview and features

Simple E-commerce Store is a complete demo online shop. Customers browse products, search, filter by category, build a cart, register or log in, check out and review their orders. Administrators manage products, categories, orders and users through the Django Admin.

Area	What is implemented
Catalogue	Categories, featured and latest products, search (name + description), category filter, sorting, pagination, product detail page, prices in ₹, stock labels, placeholder when an image is missing
Cart	Session-based. Add, update, remove, navbar item count, empty-cart state, stock validation. Prices and totals are always read from the database
Accounts	Registration, login, logout via Django's authentication system; PBKDF2 hashed passwords; password validators
Checkout	Login required. Shipping form, server-side validation, transactional order creation, stock reduction, cart cleared, confirmation page. **Demo only: simulated Cash on Delivery**
Orders	Order history, order detail (items + shipping info). Customers can only see their own orders
Admin	Products, categories, orders (status editable; totals and snapshots read-only), order items, users
Quality	38 automated tests, CSRF protection, template auto-escaping, accessible labelled forms, responsive layout

3. Hardware and software prerequisites

Requirement	Details
Operating system	Windows 11 (Windows 10 also works)
Hardware	Any modern PC: 4 GB RAM, about 500 MB free disk space
Python	3.10, 3.11, 3.12 or 3.13 (packaged and tested with Python 3.12.3)
Packages	Django 5.2.18 and Pillow 12.1.1 (pinned in requirements.txt)
Editor	Visual Studio Code with the Microsoft *Python* extension (recommended)
Browser	Chrome, Edge or Firefox
Internet	Needed once to download Python and the packages. The site itself runs fully offline
Optional	DB Browser for SQLite (to view the database), Git

4. Python installation and verification

- Download Python 3 from **python.org/downloads** (choose a 3.12 or 3.13 Windows installer).
- Run the installer and **tick “Add python.exe to PATH”** on the first screen, then click *Install Now*.
- Close and reopen any terminal windows so the new PATH is picked up.

Verify in Command Prompt or PowerShell:

```
python --version
python -m pip --version
```

Expected result: a line such as `Python 3.12.x` and a line such as `pip 24.x from C:\...\site-packages\pip (python 3.12)`.

If `python` is not recognised, try `py --version` (the Windows launcher). If that works, use `py` in place of `python` for the first command (`py -m venv .venv`). Inside the activated virtual environment `python` will work.

5. VS Code setup

- Install VS Code from **code.visualstudio.com**.
- Open the *Extensions* panel (Ctrl+Shift+X) and install **Python** (Microsoft). Optionally install **SQLite Viewer** and **Django** (Baptiste Darthenay) for template highlighting.
- Choose **File > Open Folder…** and select the extracted project folder (the one that contains `manage.py`).
- Open a terminal with **Terminal > New Terminal** (Ctrl+`). It opens in the project root.
- After creating the virtual environment (section 7), press Ctrl+Shift+P, choose **Python: Select Interpreter** and pick the one inside `.venv`.

Extraction location used in this guide: `C:\my work\WEB DEVELOPMENT\CODEALPHA INTERNSHIP POJECT`. The ZIP contains one folder, `simple_ecommerce_store`, so your project ends up at `...\CODEALPHA INTERNSHIP POJECT\simple_ecommerce_store\manage.py`. The path contains spaces, so always wrap it in double quotes in commands. No path is hard-coded in the code, so you can move the folder anywhere.

Extracting the ZIP without creating a duplicate folder: in File Explorer, right-click the ZIP > *Extract All…* and, in the dialog, **replace** the suggested destination with the target folder above (otherwise Windows adds an extra `Simple_Ecommerce_Store_Django` folder). Or use Command Prompt:

```
tar -xf Simple_Ecommerce_Store_Django.zip -C "C:\my work\WEB DEVELOPMENT\CODEALPHA INTERNSHIP POJECT"
cd /d "C:\my work\WEB DEVELOPMENT\CODEALPHA INTERNSHIP POJECT\simple_ecommerce_store"
```

6. Project folder structure

Generated from the real project files at packaging time (cache folders, the database and the virtual environment are not part of the ZIP):

```
simple_ecommerce_store/
|-- docs/
|   `-- SETUP_AND_DATABASE_GUIDE.pdf
|-- ecommerce_project/
|   |-- __init__.py
|   |-- asgi.py
|   |-- settings.py
|   |-- urls.py
|   `-- wsgi.py
|-- media/
|   `-- products/
|-- scripts/
|   |-- create_admin.bat
|   |-- run_windows.bat
|   `-- setup_windows.bat
|-- static/
|   |-- css/
|   |   `-- style.css
|   |-- images/
|   |   |-- logo.svg
|   |   `-- placeholder.svg
|   `-- js/
|       `-- main.js
|-- store/
|   |-- management/
|   |   |-- commands/
|   |   |   |-- __init__.py
|   |   |   `-- seed_demo_data.py
|   |   `-- __init__.py
|   |-- migrations/
|   |   |-- 0001_initial.py
|   |   `-- __init__.py
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- cart.py
|   |-- context_processors.py
|   |-- forms.py
|   |-- models.py
|   |-- services.py
|   |-- tests.py
|   |-- urls.py
|   `-- views.py
|-- templates/
|   |-- includes/
|   |   |-- form_fields.html
|   |   |-- order_items_table.html
|   |   `-- product_card.html
|   |-- registration/
|   |   `-- login.html
|   |-- 404.html
|   |-- 500.html
|   |-- base.html
|   |-- cart.html
|   |-- checkout.html
|   |-- home.html
|   |-- order_confirmation.html
|   |-- order_detail.html
|   |-- order_history.html
|   |-- product_detail.html
|   |-- product_list.html
|   `-- register.html
|-- .env.example
|-- .gitignore
|-- LICENSE
|-- manage.py
|-- README.md
`-- requirements.txt
```

Folder / file	Purpose
manage.py	Command-line entry point (runserver, migrate, test, ...)
ecommerce_project/	Project configuration: settings.py, root urls.py, wsgi.py / asgi.py
store/	The shop app: models, views, forms, cart logic (cart.py), order logic (services.py), admin, tests, migrations
store/management/commands/seed_demo_data.py	Creates demo categories, 12 products and generated images (safe to repeat)
templates/	HTML pages (base.html + one template per page); templates/includes holds reusable pieces
static/	Your CSS (css/style.css), JavaScript (js/main.js), logo and placeholder image
media/products/	Uploaded or generated product images
scripts/	Windows helper scripts: setup, run, create admin (they never delete anything)

7. Virtual environment creation and activation

A **virtual environment** is a private folder of Python packages for this project, so it cannot clash with other projects. Run these in the project root (the folder containing `manage.py`).

Create it (same in both shells)

```
python -m venv .venv
```

Activate it

Shell	Command
Command Prompt (cmd)	.venv\Scripts\activate
PowerShell	.venv\Scripts\Activate.ps1

Expected result: the prompt starts with `(.venv)`.

PowerShell "running scripts is disabled" error: run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, answer *Y*, then activate again. Or simply use Command Prompt.

You must activate the environment again in every new terminal window. If the prompt has no `(.venv)`, it is not active. Leave it with `deactivate`.

8. Installing Django, Pillow and other dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

requirements.txt contains exactly two packages (Django pulls in its own small helpers automatically):

```
Django==5.2.18
Pillow==12.1.1
```

Expected result: the last line says `Successfully installed ... Django-5.2.18 ... pillow-12.1.1 ...`. Check with:

```
python -m pip list
python -m django --version
```

9. SQLite database explanation

SQLite is a complete SQL database stored in a *single file*. There is no server to install or start. Django uses it by default, which makes it ideal for learning and local development.

- **Where is it?** `db.sqlite3` is created in the project root, next to `manage.py`, the first time you run `python manage.py migrate`.
- **Configured in** `ecommerce_project/settings.py` under DATABASES (engine `django.db.backends.sqlite3`, name `BASE_DIR / 'db.sqlite3'`).
- **It is not in the ZIP.** Migrations plus `seed_demo_data` rebuild everything from scratch, and no personal data is shipped.
- **Limits:** fine for one developer or a small demo; for a busy public site use PostgreSQL or MySQL.

Actual tables created for this project (Django names them <app>_<model>, so the app name *store* is the prefix):

Table	Stores	Key columns
store_category	Product categories	id, name, slug, description
store_product	Products	id, name, slug, description, price, image, category_id, stock, is_active, is_featured, created_at, updated_at
auth_user	Customer and admin accounts (Django built-in)	id, username, email, password (hashed), is_staff, is_superuser
store_order	Orders	id, order_number, user_id, customer_name, email, phone, address, city, state, postal_code, total, status, created_at
store_orderitem	Lines of each order	id, order_id, product_id, product_name, unit_price, quantity, line_subtotal

Other tables (`django_session` holds the shopping carts, `auth_group`, `django_migrations`, ...) are created and managed by Django itself.

10. Django models and the ORM

The **ORM** (Object-Relational Mapper) lets you work with database rows as Python objects instead of writing SQL by hand. A **model** is a Python class in `store/models.py`; each class becomes a table and each attribute a column. Django translates ORM calls into parameterised SQL, which also protects against SQL injection.

```
# store/models.py (shortened)
class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    category = models.ForeignKey(Category, null=True, on_delete=models.SET_NULL)

# ORM query                                    # SQL Django runs
Product.objects.filter(stock__lte=5)           # SELECT ... WHERE stock <= 5
```

Model	Relationships and notes
Category	One category has many products
Product	Belongs to a Category (deleting a category keeps the product). is_active hides a product from the store. Check constraint: price >= 0

Model	Relationships and notes
Order	Belongs to a User (PROTECT: a user with orders cannot be deleted by accident). order_number like ORD-1BFCBB80. Check constraint: total >= 0
OrderItem	Belongs to an Order. Stores a **snapshot** of product_name and unit_price, so later price or name changes never alter old orders

Try the ORM interactively with `python manage.py shell`:

```
>>> from store.models import Product
>>> Product.objects.count()
12
>>> Product.objects.filter(stock=0).values_list("name", flat=True)
<QuerySet ['Adjustable Dumbbell Pair']>
```

11. Database migration instructions

A **migration** is a versioned recipe that creates or changes tables. The ready-made file is `store/migrations/0001_initial.py`. Apply all migrations (including Django's own):

```
python manage.py migrate
```

Expected result: a list of `Applying ... OK` lines ending with `Applying store.0001_initial... OK`, and a new `db.sqlite3` file. Running it again is harmless ("No migrations to apply").

```
python manage.py showmigrations store      # [X] 0001_initial = applied
python manage.py sqlmigrate store 0001           # print the SQL it runs
python manage.py check                           # System check: no issues
```

If you change models.py yourself

```
python manage.py makemigrations
python manage.py migrate
```

Never create these tables by hand with SQL: Django will not know about them. Always change models, then makemigrations and migrate.

12. Creating the administrator account

```
python manage.py createsuperuser
```

Django asks for a username, an optional email and a password (typed twice; nothing is displayed while you type the password). Choose your own password; **no admin password is included in the project**. Alternatively run `scripts\create_admin.bat`.

Expected result: `Superuser created successfully.` Forgot it later? Run `python manage.py changepassword YOUR_USERNAME`.

13. Loading sample products

```
python manage.py seed_demo_data
```

Expected result (first run, verified at packaging time):

```
Demo data ready: 12 product(s) created, 0 already existed, 12 image(s) generated.
```

It creates **5 categories** (Electronics, Fashion, Home & Kitchen, Books, Sports & Fitness) and **12 products** with prices in rupees and different stock levels (one product is out of stock and one has only 3 left, so you can see those labels). Product pictures are drawn locally with Pillow into `media/products/`: no downloads needed.

- **Safe to run again:** products are matched by their slug, so nothing is duplicated (second run: `0 product(s) created, 12 already existed`). Prices or stock you edited in the admin are not overwritten.
- `python manage.py seed_demo_data --no-images` skips image generation.

14. Starting and accessing the website

```
python manage.py runserver
```

Expected result:

```
System check identified no issues (0 silenced).
Django version 5.2.18, using settings 'ecommerce_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Page	URL
Store home page	http://127.0.0.1:8000/
All products	http://127.0.0.1:8000/products/
Cart	http://127.0.0.1:8000/cart/
Register / Login	http://127.0.0.1:8000/register/ http://127.0.0.1:8000/login/
My orders (login needed)	http://127.0.0.1:8000/orders/
Admin panel	http://127.0.0.1:8000/admin/

Keep this terminal open while you use the site. If port 8000 is busy, use `python manage.py runserver 8001` and change the number in the URLs.

15. Using the website as a customer

- **Browse:** on the home page use the featured / latest sections or the category links in the top bar.
- **Search and filter:** type in the search box (matches names and descriptions); on *Shop* use the Category and Sort drop-downs.
- **Product page:** click a product for details, stock and a quantity selector, then *Add to Cart*. The navbar badge shows the number of items.
- **Cart:** change a quantity and press *Update* (0 removes the item), or *Remove*. You cannot add more than the stock available.
- **Register:** *Register* > choose username, email and a strong password. You are logged in automatically.
- **Checkout:** *Proceed to checkout* > fill in name, email, phone, address, city, state and a 6-digit PIN code > *Place order*. The page is labelled a demo checkout (pay on delivery, simulated).
- **Confirmation:** you see your order number (like ORD-1BFCBB80). The cart is emptied and stock is reduced.
- **History:** *My Orders* lists your orders; click one to see its items and shipping details.

16. Using Django Admin as an administrator

Open **http://127.0.0.1:8000/admin/** and log in with the account from section 12.

Admin section	What you can do
Products	Add, edit, delete. Upload an image, set price, stock, category, Active and Featured. The list lets you edit price, stock and flags inline; search by name or description; filter by category and status
Categories	Add or rename categories (the slug is filled in automatically)
Orders	Search by order number, name, email, phone or username; filter by status/date. Only **Status** can be changed (Pending, Confirmed, Shipped, Delivered, Cancelled). Customer details, totals and items are read-only; orders cannot be added or deleted here, which protects historical data
Order items	Read-only list of every purchased line with its price snapshot
Users	Django's standard user management (reset passwords, give staff access)

Setting an order to *Cancelled* does **not** automatically return its stock. If you want the units back, increase the product's stock manually.

17. Product image upload and media configuration

Product images are uploaded through the admin (*Products* > *Add product* > *Image*) and stored in `media/products/`. Relevant settings in `settings.py`:

```
MEDIA_URL  = "/media/"
MEDIA_ROOT = BASE_DIR / "media"          # <project>/media
```

- **Serving:** while `DEBUG=True`, `ecommerce_project/urls.py` serves `/media/...` for you. In production your web server must do this.
- **Validation:** only `jpg, jpeg, png, webp, gif` up to **2 MB**; the admin also checks with Pillow that the file really is an image (a renamed text file is rejected).
- **Missing image:** templates show `static/images/placeholder.svg`, so a product without a picture never breaks the page.
- **Back up** the `media/` folder together with the database (section 20).

18. SQL examples (products, users, orders, order items)

These queries use the **real table names** of this project and were executed against a database created from the project's migrations, seeded with the demo data, after one test order (2 × Wireless Earbuds). Run them in DB Browser for SQLite (*Execute SQL* tab) or the `sqlite3` command line. Use **SELECT** only unless you really know what you are doing: editing orders or stock with raw SQL bypasses all of the app's safety checks.

List the project's tables

```
SELECT name
  FROM sqlite_master
  WHERE type='table' AND (name LIKE 'store_%' OR name='auth_user')
  ORDER BY name;
```

```
name
---------------
auth_user
store_category
store_order
store_orderitem
store_product
```

Products

```
SELECT id, name, price, stock
  FROM store_product
  ORDER BY id LIMIT 4;
```

```
id | name                   | price | stock
---+------------------------+-------+------
1  | Wireless Earbuds       | 1999  | 23
2  | Smart Fitness Band     | 2499  | 18
3  | Power Bank 10000 mAh   | 1299  | 40
4  | Classic Cotton T-Shirt | 499   | 60
```

Low-stock products

```
SELECT name, stock
  FROM store_product
  WHERE stock <= 5
  ORDER BY stock;
```

```
name                     | stock
-------------------------+------
Adjustable Dumbbell Pair | 0
Web Development Handbook | 3
```

Products per category (JOIN)

```
SELECT c.name, COUNT(p.id) AS products
  FROM store_category c
  LEFT JOIN store_product p ON p.category_id = c.id
  GROUP BY c.name
  ORDER BY c.name;
```

```
name             | products
-----------------+---------
Books            | 2
Electronics      | 3
Fashion          | 3
Home & Kitchen   | 2
Sports & Fitness | 2
```

Users

```
SELECT id, username, email, is_staff
  FROM auth_user;
```

```
id | username | email         | is_staff
---+----------+---------------+---------
1  | curluser | c@example.com | 0
```

The `password` column holds a salted hash like `pbkdf2_sha256$...`, never the real password.

Orders

```
SELECT order_number, customer_name, total, status
  FROM store_order
  ORDER BY created_at DESC;
```

```
order_number | customer_name | total | status
-------------+---------------+-------+--------
ORD-1BFCBB80 | Test User     | 3998  | pending
```

Order items (JOIN orders and items)

```
SELECT o.order_number, i.product_name, i.unit_price, i.quantity, i.line_subtotal
  FROM store_orderitem i
  JOIN store_order o ON o.id = i.order_id;
```

```
order_number | product_name     | unit_price | quantity | line_subtotal
-------------+------------------+------------+----------+--------------
ORD-1BFCBB80 | Wireless Earbuds | 1999       | 2        | 3998
```

Order count and revenue

```
SELECT COUNT(*) AS orders, SUM(total) AS revenue
  FROM store_order
  WHERE status <> 'cancelled';
```

```
orders | revenue
-------+--------
1      | 3998
```

SQLite stores decimals as plain numbers, so a price of 1999.00 is displayed as 1999 here; the website formats it as ₹1,999.00. Your own output will differ (different order numbers, users and quantities); the columns will be the same. Stock for Wireless Earbuds dropped from 25 to 23 after that test order.

19. How to use DB Browser for SQLite

- Download **DB Browser for SQLite** (sqlitebrowser.org) and install it. It is optional.
- Open it, choose **File > Open Database**, set the file filter to *All files* and select `db.sqlite3` in the project root.
- **Database Structure** tab: the tables listed in section 9. **Browse Data** tab: pick a table to see its rows.
- **Execute SQL** tab: paste a query from section 18 and press the play button (Ctrl+Return).
- To only look at data, open with **File > Open Database Read Only**.

If you edit data in DB Browser you must click **Write Changes** (Ctrl+S). While it holds unsaved changes Django may show *database is locked*. Close DB Browser (or press Write/Revert) before using the website.

20. Backing up and restoring the development database

Back up `db.sqlite3` **and** the `media/products/` images before anything destructive (deleting the database, running `flush`, changing models heavily, reinstalling).

Back up (stop the server with Ctrl+C first)

```
mkdir backups
copy db.sqlite3 backups\db-backup.sqlite3
xcopy media backups\media /E /I /Y
```

Safer while the server is running, because SQLite makes a consistent copy (works in both shells):

```
python -c "import sqlite3; s=sqlite3.connect('db.sqlite3'); d=sqlite3.connect('backups/db-backup.sqlite3'); s.backup(
```

Optional text backup of the data as JSON:

```
python manage.py dumpdata store auth.user --indent 2 --output backups\data.json
```

Restore

```
copy /Y backups\db-backup.sqlite3 db.sqlite3
xcopy backups\media media /E /I /Y
```

PowerShell equivalents: `Copy-Item db.sqlite3 backups\db-backup.sqlite3` and `Copy-Item backups\db-backup.sqlite3 db.sqlite3 -Force`.

Start completely fresh (destructive: back up first!)

Stop the server, delete `db.sqlite3`, then run `python manage.py migrate`, `createsuperuser` and `seed_demo_data` again. The provided scripts never do this for you.

The `backups/` folder is listed in `.gitignore`, so backups are not accidentally published.

21. Running the automated tests

```
python manage.py test
```

Django creates a temporary in-memory test database, so your real `db.sqlite3` is never touched. **Result when this guide was produced** (Python 3.12.3, Django 5.2.18):

```
Ran 38 tests in ~12s

OK
```

Test group	Covers
CatalogueTests	Homepage, listing, detail, search, category filter, out-of-stock label, placeholder image, unique slugs
AuthTests	Registration (+ password hashing), validation errors, login, logout, bad login
CartTests	Add/update/remove, totals, navbar count, persistence, invalid quantities, stock limits, tampered session, open-redirect protection
CheckoutTests	Login protection, empty cart, successful order, order items, stock reduction, cart clearing, price snapshots, forged totals ignored, failure and rollback, other users' orders hidden
AdminTests	Admin pages load, product creation, order protection (read-only fields, no add/delete)
ImageValidationTests / SeedCommandTests	Bad extension and fake image rejected; demo data command is repeatable

Other useful commands: `python manage.py check` (system check) and `python manage.py makemigrations --check --dry-run` (verifies migrations match the models; expect “No changes detected”). Run a single group with `python manage.py test store.tests.CartTests`.

22. Troubleshooting common errors

Symptom	Cause and fix
'python' is not recognized...	Python is not on PATH. Reinstall and tick *Add python.exe to PATH*, restart the terminal, or use `py`.
Activate.ps1 cannot be loaded because running scripts is disabled	PowerShell policy. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or use Command Prompt.
ModuleNotFoundError: No module named 'django'	Virtual environment not active or packages not installed. Activate it (no `(.venv)` in the prompt?) and run `python -m pip install -r requirements.txt`.
Couldn't import Django (from manage.py)	Same as above.
OperationalError: no such table: store_product	Migrations not applied. Run `python manage.py migrate`.
Cannot use ImageField because Pillow is not installed	Run `python -m pip install -r requirements.txt` inside the active venv.
Error: That port is already in use	Another server is running. Close it (Ctrl+C) or use `python manage.py runserver 8001`.
Pillow / pip fails to install on a very new Python	Use Python 3.10 - 3.13. Newer versions may need different pinned package versions.
Images do not appear	Run `seed_demo_data` or upload via admin; media is only served when `DJANGO_DEBUG` is True (default locally). Check the file exists in `media/products/`.
403 CSRF verification failed	Open the site with the same host you logged in on (127.0.0.1 vs localhost), enable cookies, refresh the page and retry.
Cannot log in to /admin/	Create the admin with `createsuperuser`, or reset with `changepassword USERNAME`. Customer accounts are not staff.
OperationalError: database is locked	Close DB Browser for SQLite or write/revert its pending changes, then retry.
ImproperlyConfigured: Set DJANGO_SECRET_KEY...	You set `DJANGO_DEBUG=False` without a secret key. Remove it for local use or define a key in `.env`.
TemplateDoesNotExist / NoReverseMatch	A file was moved or deleted. Restore it from the ZIP; do not rename the templates or URL names.
Commands fail because of the folder name	The path has spaces. Wrap it in quotes: `cd /d "C:\my work\WEB DEVELOPMENT\..."`

23. Windows Command Prompt vs PowerShell

Task	Command Prompt (cmd)	PowerShell
Activate venv	.venv\Scripts\activate	.venv\Scripts\Activate.ps1
Leave venv	deactivate	deactivate
Change drive + folder	cd /d "D:\folder name"	cd "D:\folder name"
Make folder	mkdir backups	mkdir backups (or New-Item -ItemType Directory backups)
Copy file	copy a.txt b.txt	Copy-Item a.txt b.txt
List files	dir	dir or ls
Set env variable (this window)	set DJANGO_DEBUG=True	$env:DJANGO_DEBUG = "True"
Run a .bat script	scripts\run_windows.bat	.\scripts\run_windows.bat
python, pip, manage.py commands	identical	identical

The VS Code terminal opens PowerShell by default. To switch: click the drop-down arrow next to the **+** in the terminal panel and choose *Command Prompt*.

24. Stopping and restarting the application

- **Stop:** click in the terminal that runs the server and press **Ctrl+C** (on some keyboards Ctrl+Break). The site becomes unreachable.
- **Restart later:** open a terminal in the project folder, **activate the virtual environment again**, then run the server:

```
cd /d "C:\my work\WEB DEVELOPMENT\CODEALPHA INTERNSHIP POJECT\simple_ecommerce_store"
.venv\Scripts\activate
python manage.py runserver
```

- Your data (products, users, orders) stays in `db.sqlite3`. You do **not** need to run migrate or seed_demo_data again unless you changed models or want the demo products back.
- Shortcut: double-click `scripts\run_windows.bat`.
- The development server restarts itself when you edit Python files (when started without `--noreload`).

25. Security notes and limitations

Implemented:

- CSRF tokens on every form and POST-only endpoints for cart changes and logout.
- Passwords hashed with Django's PBKDF2 and checked by the four default validators.
- Template auto-escaping of all user-supplied text; ORM queries only (no string-built SQL).
- Prices and totals computed on the server from the database; the cart session stores only ids and quantities and is sanitised on every request.
- Order creation inside one database transaction with row locking where the database supports it; stock updates use an atomic `stock >= qty` check.
- Orders are fetched with the logged-in user as part of the query, so another user's order returns 404.
- Redirect targets (`next`) are validated to prevent open redirects; uploads are limited by type, size and image content.
- SECRET_KEY, DEBUG and ALLOWED_HOSTS come from environment variables / a git-ignored `.env` file.

Limitations (read this):

- **Demo checkout only.** There is no payment gateway; nothing is charged. Do not describe it as processing real payments.
- **Not production-ready as shipped.** The defaults are for local development.
- SQLite serialises writes, so it is unsuitable for heavy concurrent traffic. `select_for_update` has no effect on SQLite (it is safe there only because SQLite locks the whole database).
- No email confirmation, password-reset emails, order-cancellation flow, tax/shipping calculation or rate limiting.
- Cancelling an order in the admin does not restock items automatically.
- The site was verified with the automated test suite and HTTP requests against the running development server. It was not clicked through in a real browser at packaging time.

Settings to change before any deployment

Setting	Required change
DJANGO_DEBUG	False
DJANGO_SECRET_KEY	A long, unique, secret random value (never commit it)
DJANGO_ALLOWED_HOSTS	Your real domain name(s)
HTTPS	Serve over HTTPS (with DEBUG off, secure cookies and SSL redirect are enabled automatically)
Static / media files	Run `python manage.py collectstatic` and have the web server serve STATIC_ROOT and MEDIA_ROOT
Database	Move to PostgreSQL or MySQL; take regular backups
Email / payments	Configure a real email backend and a real payment provider

26. Final verification checklist

Tick each item to confirm your installation works:

- ☐ python --version and python -m pip --version both print versions
- ☐ Project opened in VS Code; terminal shows the project root (manage.py is visible)
- ☐ (.venv) appears in the prompt after activation
- ☐ pip install -r requirements.txt finished without errors
- ☐ python manage.py migrate ends with 'Applying store.0001_initial... OK' and db.sqlite3 exists
- ☐ python manage.py check reports 'no issues'
- ☐ python manage.py createsuperuser succeeded
- ☐ python manage.py seed_demo_data reports 12 products
- ☐ python manage.py test prints 'Ran 38 tests ... OK'
- ☐ python manage.py runserver starts; http://127.0.0.1:8000/ shows the store with 12 products
- ☐ http://127.0.0.1:8000/admin/ lets you log in and shows Products, Categories, Orders
- ☐ Search 'yoga' finds the yoga mat; the Books category filter shows 2 products
- ☐ Adding products updates the cart badge; updating and removing items works
- ☐ Adding more than the stock available is refused with an error message
- ☐ Checkout while logged out redirects to the login page
- ☐ A registered customer can place an order and sees the confirmation page
- ☐ The order appears under My Orders and in Admin > Orders; stock decreased
- ☐ Another user cannot open your order URL (404)
- ☐ Ctrl+C stops the server; you can restart it after re-activating the venv

Project files are for learning and demonstration. See README.md and LICENSE for usage notes.